DSA 0102 – OBJECT ORIENTED PROGRAMMING IN C++

Name: Sanjay Manesh M

Reg No: 192511362

ASSIGNMENT 4

Q 1 – Class Design Review: Diagnose and Repair Duplicated Classes

Introduction

Class design is an important part of object-oriented programming. A good class hierarchy reduces code duplication, improves reusability and makes programs easier to maintain. In the given college payroll system, three separate classes named `Professor`, `LabAssistant` and `GuestFaculty` contain some repeated information and payment-related operations. These classes can be redesigned using inheritance to create a common base class and separate derived classes.

Design Deficiencies

The original design contains several problems. The `Professor` and `LabAssistant` classes have repeated data members such as `name`, `id` and `basic`. Both classes also use the same payment calculation, which increases code duplication. All data members are declared public, allowing them to be accessed and modified directly without proper control. The `GuestFaculty` class uses different names such as `nm` and `code` for information that represents the same details as `name` and `id` in other classes.

The `da` data member is declared but never used in the payment calculation. The value `0.42` is directly written in the payment formula, making it a magic number. If the payment rate changes, multiple classes may need modification. Adding a new staff category would also require creating and maintaining another separate class. Furthermore, the program cannot conveniently process all staff objects through a common base-class pointer.

Redesigned Class Hierarchy

The improved design introduces an abstract base class called `Staff`, which stores the common information such as employee name and ID. It also declares a virtual `pay()` function that must be implemented by its derived classes.

The `Professor`, `LabAssistant` and `GuestFaculty` classes inherit publicly from the `Staff` class. Each derived class implements its own version of the `pay()` function. Professors receive basic pay with an additional research allowance, lab assistants receive basic

pay with the standard allowance, and guest faculty members are paid according to their working hours and hourly rate.

A common helper function is used to calculate basic pay with the allowance, avoiding repeated payment formulas. The allowance rate is defined using a named constant instead of directly using a magic number.

Design Defence

The redesigned hierarchy stores common information in the `Staff` base class and allows each derived class to implement its own payment calculation. When a new staff category is introduced, a new derived class can be added by implementing `pay()` without changing the existing classes. This improves reusability, reduces duplication and supports runtime polymorphism through base-class pointers.

Alternative Concept and Complexity Analysis

An alternative design is to create a single `Staff` class containing a type field to identify whether the employee is a professor, lab assistant or guest faculty member. A `switch` statement inside the `pay()` function can calculate the payment based on the employee type.

Both approaches require O(1) time for an individual payment calculation. However, the single-class approach may contain unused data members because different staff categories require different information. When a new staff type is added, the type field, payment switch and related class logic must be modified. In the inheritance approach, a new derived class can be created without modifying the existing payment implementations. Therefore, inheritance provides better extensibility and maintainability.

Conclusion

The redesigned staff hierarchy solves the problems of duplicated data, repeated payment calculations and poor data protection. By using inheritance, virtual functions and runtime polymorphism, the payroll system becomes more reusable and easier to extend. This design also allows different staff categories to be processed through a common base-class interface.

Program Code

```
#include <iostream>

#include <string>

#include <vector>

#include <memory>

#include <iomanip>



using namespace std;
```

```
// Base class
class Staff {
protected:
    string name;
    int id;

public:
    Staff(string n, int i) {
        name = n;
        id = i;
    }

    virtual double pay() const = 0;

    virtual void display() const {
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
        cout << "Pay: Rs " << fixed << setprecision(2)
             << pay() << endl;
        cout << "----------------------" << endl;
    }

    virtual ~Staff() {}
};
```

```
// Derived class: Professor
class Professor : public Staff {
private:
    double basic;
    double research;

public:
    Professor(string n, int i, double b, double r)
        : Staff(n, i) {
        basic = b;
        research = r;
    }

    double pay() const override {
        return basic + basic * 0.42 + research;
    }
};

// Derived class: Lab Assistant
class LabAssistant : public Staff {
private:
    double basic;

public:
    LabAssistant(string n, int i, double b)
        : Staff(n, i) {
```

```
        basic = b;
    }

    double pay() const override {
        return basic + basic * 0.42;
    }
};

// Derived class: Guest Faculty
class GuestFaculty : public Staff {
private:
    double hours;
    double rate;

public:
    GuestFaculty(string n, int i, double h, double r)
        : Staff(n, i) {
        hours = h;
        rate = r;
    }

    double pay() const override {
        return hours * rate;
    }
};
```

```
int main() {
    vector<unique_ptr<Staff>> staff;

    // Creating objects without make_unique()
    staff.push_back(
        unique_ptr<Staff>(new Professor("Arun", 101, 50000, 10000))
    );

    staff.push_back(
        unique_ptr<Staff>(new LabAssistant("Kumar", 102, 30000))
    );

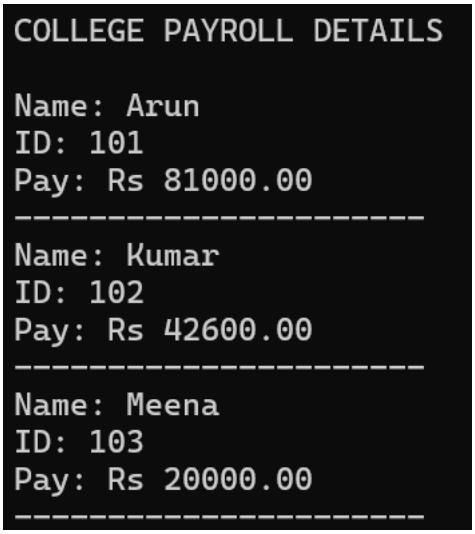
    staff.push_back(
        unique_ptr<Staff>(new GuestFaculty("Meena", 103, 40, 500))
    );

    cout << "COLLEGE PAYROLL DETAILS" << endl;
    cout << endl;

    for (const auto& s : staff) {
        s->display();
    }

    return 0;
}
```

Output



QUESTION 2 – FROM REQUIREMENT TO HIERARCHY

2.1 Identify the Relationships

The university maintains information about persons, students, staff members, teaching assistants and research scholars. These entities can be organized using inheritance to reduce code duplication and improve reusability.

Relationship	Type	Explanation
Student inherits Person	Is-a	A student is a person.
Staff inherits Person	Is-a	A staff member is a person.
TeachingAssistant inherits Student and Staff	Is-a	A teaching assistant performs both roles.
ResearchScholar inherits Student	Is-a	A research scholar is a specialized student.

Types of inheritance used:

1. Single inheritance: Student inherits from Person.
2. Hierarchical inheritance: Student and Staff inherit from Person.
3. Multiple inheritance: TeachingAssistant inherits from Student and Staff.
4. Multilevel inheritance: Person → Student → ResearchScholar.
5. Hybrid inheritance: The complete design combines multiple types of inheritance.

2.2 Resolve the Diamond Problem

The diamond problem occurs when a class inherits from two classes that share the same base class.

In this scenario, TeachingAssistant inherits from both Student and Staff, and both inherit from Person. Without virtual inheritance, TeachingAssistant receives two separate Person

subobjects. This creates duplicate data and ambiguity when accessing common members such as name and ID.

Solution – Virtual Base Class . Virtual inheritance ensures that only one shared Person subobject is created.

2.3 Design the Classes

Public inheritance is used because Student and Staff are types of Person, while TeachingAssistant is both a Student and a Staff member. Protected data members allow derived classes to access common information without making it directly accessible outside the class.

The hierarchy contains the following classes:

- Person: Stores name, ID and phone number.
- Student: Stores programme and CGPA.
- Staff: Stores department and salary.
- TeachingAssistant: Combines Student and Staff through multiple inheritance.
- ResearchScholar: Extends Student by adding fellowship information.

2.4 Access Table

The following table explains how the members of Person are inherited by Student under different inheritance modes.

Person member	Public inheritance	Protected inheritance	Private inheritance
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible	Not accessible	Not accessible

Conclusion

Inheritance helps organize university records by reusing common Person information. Virtual inheritance solves the diamond problem by avoiding duplicate Person subobjects. Therefore, the proposed hierarchy improves code reusability, reduces duplication and supports future extensions.

Program Code 1:

```
#include <iostream>

#include <string>

using namespace std;



class Person {
```

```
protected:
    string name;
    int id;
    string phone;

public:
    Person(string n, int i, string p)
        : name(n), id(i), phone(p) {}
};

class Student : virtual public Person {
protected:
    string programme;
    double cgpa;

public:
    Student(string n, int i, string p,
            string prog, double c)
        : Person(n, i, p),
          programme(prog), cgpa(c) {}
};

class Staff : virtual public Person {
protected:
    string department;
    double salary;
```

```
public:
    Staff(string n, int i, string p,
          string dept, double s)
        : Person(n, i, p),
          department(dept), salary(s) {}
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(string n, int i, string p,
                      string prog, double c,
                      string dept, double s)
        : Person(n, i, p),
          Student(n, i, p, prog, c),
          Staff(n, i, p, dept, s) {}

    void display() const {
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
        cout << "Phone: " << phone << endl;
        cout << "Programme: " << programme << endl;
        cout << "CGPA: " << cgpa << endl;
        cout << "Department: " << department << endl;
        cout << "Salary: " << salary << endl;
    }
```

```
};

class ResearchScholar : public Student {
private:
    double fellowship;

public:
    ResearchScholar(string n, int i, string p,
                    string prog, double c, double f)
        : Person(n, i, p),
          Student(n, i, p, prog, c),
          fellowship(f) {}

    void display() const {
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
        cout << "Programme: " << programme << endl;
        cout << "CGPA: " << cgpa << endl;
        cout << "Fellowship: Rs " << fellowship << endl;
    }
};

int main() {
    TeachingAssistant ta(
        "Arun", 101, "9876543210",
        "MCA", 8.5, "Computer Science", 30000
```

```
    );

    ResearchScholar rs(
        "Kavya", 102, "9876501234",
        "PhD", 9.2, 25000
    );

    cout << "TEACHING ASSISTANT DETAILS" << endl;
    ta.display();

    cout << "\nRESEARCH SCHOLAR DETAILS" << endl;
    rs.display();

    return 0;
}
```

Output

```
TEACHING ASSISTANT DETAILS
Name: Arun
ID: 101
Phone: 9876543210
Programme: MCA
CGPA: 8.5
Department: Computer Science
Salary: 30000

RESEARCH SCHOLAR DETAILS
Name: Kavya
ID: 102
Programme: PhD
CGPA: 9.2
Fellowship: Rs 25000
```

Program Code 2:

```
#include <iostream>
using namespace std;

struct Person {
    Person() {
        cout << "Person ";
    }
};

struct Student : virtual Person {
    Student() {
        cout << "Student ";
    }
};

struct Staff : virtual Person {
    Staff() {
        cout << "Staff ";
    }
};

struct TeachingAssistant : Student, Staff {
    TeachingAssistant() {
        cout << "TA ";
```

```
    }
};

int main() {
    TeachingAssistant ta;
    cout << '\n';

    return 0;
}
```

Output

```
Person Student Staff TA
```

QUESTION 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER CLASSES

3.1 Pass Values to Base Constructors

In C++, a derived class constructor can pass values to its base class constructor using a member initializer list. This ensures that base-class members are initialized before the derived-class members.

In the university hierarchy, Student and Staff inherit Person virtually. When a TeachingAssistant object is created, the most-derived class, TeachingAssistant, must initialize the shared virtual Person base directly.

The constructors use the following initialization approach:

- Student initializes Person and its own student details.
- Staff initializes Person and its own staff details.
- TeachingAssistant initializes the shared Person base and then initializes Student and Staff.

Reason: When virtual inheritance is used, the most-derived class is responsible for initializing the virtual base class. The initializers written in Student and Staff are ignored when a TeachingAssistant object is created.

3.2 Member Classes

A member class is a class whose object is included as a data member inside another class. This represents a has-a relationship.

In this task:

- Person has an Address object.
- Student has a Course object.

The Address class stores the city and PIN code, while the Course class stores course information. Their constructors receive values through the constructors of the containing classes.

Using member classes improves code organization and allows the same classes to be reused in other parts of the application.

3.3 Trace the Order of Construction and Destruction

C++ follows a specific order when constructing and destroying objects.

Construction order:

1. Virtual base classes are constructed first.
2. Other base classes are constructed next.
3. Member objects are constructed in their declaration order.
4. The derived-class constructor body executes last.

Destruction order:

Destruction happens in the reverse order of construction. The derived-class destructor executes first, followed by member objects and base classes in reverse order.

Construction and destruction observations:

Object	**Construction order**	**Destruction order**
Student	Person, Course, Student	Student, Course, Person
Staff	Person, Staff	Staff, Person
TeachingAssistant	Person, Student, Staff, TeachingAssistant	TeachingAssistant, Staff, Student, Person

The Address member is constructed before the Person constructor body and destroyed after the Person destructor body.

3.4 Experiment – Removing Virtual Inheritance

When the virtual keyword is removed from Student and Staff, each class receives a separate Person subobject.

With virtual inheritance: Person Student Staff TA

Without virtual inheritance: Person Student Person Staff TA

The Person constructor executes twice without virtual inheritance because Student and Staff have separate Person base objects.

The size of TeachingAssistant may also increase because of the duplicate Person subobject. The exact size depends on the compiler, system architecture and memory alignment. It must be measured using sizeof().

Complete the Code – TODO 1

The Student constructor must initialize the Person base class, registration number and Address member through a member initializer list.

Expected output:

192311001 Kavya (19) Salem-636001

The output confirms that all values have been initialized correctly and that the Address member is accessible through the Student object.

Initializer List Order Experiment

If the order of elements in the initializer list is changed, the actual initialization order does not change.

C++ initializes:

1. Base classes first.
2. Data members in the order in which they are declared in the class.

When compiled with -Wall -Wextra, the compiler may generate a -Wreorder warning if the initializer list order differs from the declaration order. The warning reminds the programmer that the textual order does not control actual initialization.

Alternative Concept and Complexity Analysis

Instead of storing Address as a direct member object, Student can store a pointer to a dynamically allocated Address object.

In this approach, the constructor must allocate memory using new, the copy constructor must perform a deep copy, and the destructor must release memory using delete.

Feature	**Address member object**	**Address pointer**
Memory allocation	Automatic	Dynamic
Extra pointer storage	Not required	Required
Allocation overhead	No separate allocation	Additional allocation
Destructor	Automatic	Manual deletion

Copy handling	Automatic member copy	Deep copy required
Access time	O(1)	O(1)

A direct Address member is simpler and safer when every Student must have an Address. A pointer is useful when the Address is optional or needs to be changed dynamically.

Conclusion

Constructors and destructors are important for managing object initialization and destruction in inheritance hierarchies. Member initializer lists ensure correct initialization of base classes and member objects. Virtual inheritance avoids duplicate base objects, while composition supports reusable member classes such as Address and Course.

Program Code1 : Constructors for Student, Staff and TeachingAssistant

```
#include <iostream>

#include <string>

using namespace std;



class Person {

protected:

    string name;

    int id;

    string phone;



public:

    Person(string n, int i, string p)

        : name(n), id(i), phone(p) {

        cout << "Person constructor\n";

    }

};



class Student : virtual public Person {
```

```
protected:
    string programme;
    double cgpa;

public:
    Student(string n, int i, string p,
            string prog, double c)
        : Person(n, i, p),
          programme(prog), cgpa(c) {
        cout << "Student constructor\n";
    }
};

class Staff : virtual public Person {
protected:
    string department;
    double salary;

public:
    Staff(string n, int i, string p,
          string dept, double s)
        : Person(n, i, p),
          department(dept), salary(s) {
        cout << "Staff constructor\n";
    }
};
```

```
class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(string n, int i, string p,
                      string prog, double c,
                      string dept, double s)
        : Person(n, i, p),
          Student(n, i, p, prog, c),
          Staff(n, i, p, dept, s) {
        cout << "TeachingAssistant constructor\n";
    }
};

int main() {
    TeachingAssistant ta(
        "Arun", 101, "9876543210",
        "MCA", 8.5, "Computer Science", 30000
    );

    return 0;
}
```

Output

```
Person constructor
Student constructor
Staff constructor
TeachingAssistant constructor
```

Program Code2 : Member Classes and Complete TODO 1

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
    int age;

public:
    Person(const string& n, int a)
        : name(n), age(a) {}
};

class Address {
    string city;
    int pin;

public:
    Address(const string& c, int p)
        : city(c), pin(p) {}

    string str() const {
        return city + "-" + to_string(pin);
    }
```

```
};

class Student : public Person {
    long long regNo;
    Address home;

public:
    // TODO 1 completed
    Student(const string& n, int a, long long r,
            const string& city, int p)
        : Person(n, a), regNo(r), home(city, p) {}

    void print() const {
        cout << regNo << ' ' << name << " ("
             << age << ") " << home.str() << '\n';
    }
};

int main() {
    Student s("Kavya", 19, 192311001, "Salem", 636001);

    s.print();

    return 0;
}
```

Output

```
192311001 Kavya (19) Salem-636001
```

Program Code3 : Construction and Destruction Order

```
#include <iostream>

using namespace std;


class Address {

public:

    Address() {

        cout << "Address ";

    }


    ~Address() {

        cout << "~Address ";

    }

};


class Course {

public:

    Course() {

        cout << "Course ";

    }


    ~Course() {

        cout << "~Course ";
```

```
    }
};

class Person {
protected:
    Address home;

public:
    Person() {
        cout << "Person ";
    }

    virtual ~Person() {
        cout << "~Person ";
    }
};

class Student : virtual public Person {
    Course course;

public:
    Student() {
        cout << "Student ";
    }

    ~Student() {
```

```
        cout << "~Student ";
    }
};

class Staff : virtual public Person {
public:
    Staff() {
        cout << "Staff ";
    }

    ~Staff() {
        cout << "~Staff ";
    }
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant() {
        cout << "TA ";
    }

    ~TeachingAssistant() {
        cout << "~TA ";
    }
};
```

```
int main() {
    cout << "Student object: ";
    {
        Student s;
    }

    cout << "\nStaff object: ";
    {
        Staff s;
    }

    cout << "\nTeachingAssistant object: ";
    {
        TeachingAssistant ta;
    }

    cout << '\n';

    return 0;
}
```

Output

```
Student object: Address Person Course Student ~Student ~Course ~Person ~Address
Staff object: Address Person Staff ~Staff ~Person ~Address
TeachingAssistant object: Address Person Course Student Staff TA ~TA ~Staff ~Student ~Course ~Person ~Address
```

Program Code4 :

```
#include <iostream>
using namespace std;

struct Engine {
    Engine() {
        cout << "Engine ";
    }

    ~Engine() {
        cout << "~Engine ";
    }
};

struct Vehicle {
    Vehicle() {
        cout << "Vehicle ";
    }

    ~Vehicle() {
        cout << "~Vehicle ";
    }
};

struct Car : Vehicle {
    Engine e;
```

```
    Car() {
        cout << "Car ";
    }

    ~Car() {
        cout << "~Car ";
    }
};

int main() {
    {
        Car c;
        cout << "| ";
    }

    cout << '\n';

    return 0;
}
```

Output

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

Program Code5 : Compare Object Size With and Without Virtual Inheritance

```
#include <iostream>
using namespace std;
```

```
class Person {
protected:
    int id;
    double phone;
};

class Student : virtual public Person {
    double cgpa;
};

class Staff : virtual public Person {
    double salary;
};

class TeachingAssistant : public Student, public Staff {
    double allowance;
};

class Student2 : public Person {
    double cgpa;
};

class Staff2 : public Person {
    double salary;
};
```

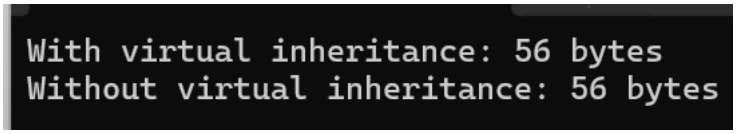
```
class TeachingAssistant2 : public Student2, public Staff2 {
    double allowance;
};

int main() {
    cout << "With virtual inheritance: "
         << sizeof(TeachingAssistant) << " bytes\n";

    cout << "Without virtual inheritance: "
         << sizeof(TeachingAssistant2) << " bytes\n";

    return 0;
}
```

Output



Program Code6 : Initializer List Order Experiment

```
#include <iostream>
using namespace std;

class Test {
    int first;
    int second;
```

```
public:
    Test() : second(20), first(10) {
        cout << first << " " << second << '\n';
    }
};

int main() {
    Test t;
    return 0;
}
```

Output

```
10 20
```

Question 4 – Pointers to Derived Classes: Forensics

4.1 Identify the Problems in the Given Program

The given program uses a base class Account and a derived class SeniorAccount. It contains several problems related to pointers, virtual functions, destructors, and object slicing.

Problems identified:

1. Non-virtual function: The rate() function in the base class is not declared virtual. Therefore, calling it through an Account* always executes the base-class version, even when the pointer refers to a SeniorAccount object.
2. Non-virtual destructor: The base-class destructor is not virtual. Deleting a derived object through a base-class pointer may not call the derived destructor, causing the dynamically allocated history array to leak.
3. Object slicing: The statement Account copy = SeniorAccount(); copies only the base-class portion of the derived object. The derived-class information is lost.

4.2 Demonstrating the Problems

When a SeniorAccount object is accessed using an Account*, the base-class function is called because rate() is non-virtual.

Similarly, when a derived object is deleted through a base pointer, the derived destructor is not reliably invoked without a virtual base destructor. This prevents proper resource cleanup.

Object slicing occurs when a derived object is assigned to a base-class object by value. Only the base-class data is retained.

4.3 Corrected Class Design

The problems can be solved using runtime polymorphism and proper destructor design.

- Declare rate() as a virtual function in the base class.
- Use override in the derived class to ensure correct overriding.
- Declare the base-class destructor as virtual.
- Use base-class pointers or references to access derived objects without slicing.
- Use smart pointers or properly managed pointers to ensure safe memory management.

4.4 Important Rules

Concept	Rule
Virtual function	Enables runtime polymorphism
Override	Confirms that a derived function overrides a base function
Virtual destructor	Ensures proper derived-class destruction
Object slicing	Avoid passing derived objects by value as base objects
Base pointer	Can point to a derived-class object

Conclusion

This task demonstrates the importance of virtual functions, virtual destructors, and avoiding object slicing. These concepts help create safe and reusable class hierarchies using runtime polymorphism.

Program Code1 : Demonstrating Static and Dynamic Binding

```
#include <iostream>
using namespace std;


class Account {
public:
   double rate() const {
      return 3.5;
   }


   virtual double vrate() const {
```

```
        return 3.5;
    }

    virtual ~Account() {}
};

class SeniorAccount : public Account {
public:
    double rate() const {
        return 4.0;
    }

    double vrate() const override {
        return 4.0;
    }
};

int main() {
    SeniorAccount s;
    Account* p = &s;

    Account copy = s;

    cout << p->rate() << " ";
    cout << p->vrate() << " ";
    cout << copy.vrate() << endl;

    return 0;
}
```

Output

```
3.5 4 3.5
```

Program Code2 : Demonstrating Virtual Destructor

```
#include <iostream>
using namespace std;

class Account {
public:
    Account() {
        cout << "Account constructor" << endl;
    }

    virtual ~Account() {
        cout << "Account destructor" << endl;
    }
};

class SeniorAccount : public Account {
    double* history;

public:
    SeniorAccount() {
        history = new double[100];
        cout << "SeniorAccount constructor" << endl;
    }

    ~SeniorAccount() override {
        delete[] history;
        cout << "SeniorAccount destructor" << endl;
    }
};

int main() {
    Account* p = new SeniorAccount();
```

```
    delete p;


    return 0;
}
```

Output:

```
Account constructor
SeniorAccount constructor
SeniorAccount destructor
Account destructor
```

Question 5 – Pointers to Objects and the this Pointer

5.1 Different Ways to Access Objects

In C++, objects can be accessed directly, through pointers, through references, or through dynamically allocated memory. The this pointer is a special pointer available inside non-static member functions. It points to the current object.

The following methods are used to access objects:

1. Direct object access: Members are accessed using the dot (.) operator.
2. Pointer access: A pointer stores the address of an object, and members are accessed using the arrow (->) operator.
3. Reference access: A reference acts as another name for an existing object.
4. Dynamic object creation: Objects can be created using new and released using delete.

5.2 Understanding the this Pointer

The this pointer refers to the current object that calls a member function.

It is useful for:

- Distinguishing class data members from function parameters with the same name.
- Returning the current object using *this.
- Supporting function chaining, where multiple member functions are called in a single statement.

5.3 Array of Pointers to Objects

An array of pointers can store the addresses of multiple customer cart objects. Each pointer can access the corresponding cart and calculate its total.

To find the cart with the highest total:

1. Create multiple cart objects.
2. Store their addresses in an array of pointers.
3. Calculate each cart's total.
4. Compare the totals and identify the highest value.

The operation takes \(O(n)\) time for \(n\) carts.

5.4 Comparison of Object Access Methods

Method	Operator	Safety and usage
Direct object	.	Simple access to an existing object
Pointer	->	Useful for indirect access
Reference	Direct member access	Cannot normally be reseated after initialization
Dynamic pointer	->	Useful when object lifetime is controlled dynamically

5.5 Why Is this a Pointer?

The this pointer stores the address of the current object. Therefore, this refers to the current object's address, while *this refers to the actual object.

For example:

- this->discount accesses the current object's discount member.
- return *this; returns the current object by reference when the function's return type is Cart&.

Conclusion

This task explains how pointers, references, and the this pointer are used to access and manage objects in C++. Function chaining using this improves usability, while arrays of pointers help manage multiple objects efficiently.

Program Code1 :

```
#include <iostream>

#include <iomanip>

#include <string>

using namespace std;



class Cart {

   string items[10];
```

```
    double prices[10];
    int n = 0;
    double discount = 0;

public:
    Cart& add(string item, double price) {
        if (n < 10) {
            items[n] = item;
            prices[n] = price;
            n++;
        }

        return *this;
    }

    Cart& setDiscount(double discount) {
        this->discount = discount;
        return *this;
    }

    double total() const {
        double t = 0;

        for (int i = 0; i < n; i++) {
            t += prices[i];
        }
```

```
        return t * (100 - discount) / 100;
    }
};

int main() {
    Cart c;

    c.add("Rice", 60)
     .add("Dal", 120)
     .setDiscount(10);

    Cart* p = &c;
    p->add("Oil", 180);

    cout << fixed << setprecision(2);
    cout << "Cart Total: Rs. " << p->total() << endl;

    return 0;
}
```

Output

```
Cart Total: Rs. 324.00
```

Program Code2 : Demonstrating Different Ways to Access Objects

```
#include <iostream>
using namespace std;
```

```
class Cart {
public:
    int total;

    Cart(int t) {
        total = t;
    }

    void display() {
        cout << "Total: " << total << endl;
    }
};

int main() {
    Cart c1(500);

    // Direct object access
    c1.display();

    // Pointer access
    Cart* p = &c1;
    p->display();

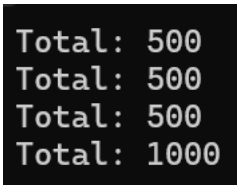
    // Reference access
    Cart& r = c1;
    r.display();

    // Dynamic object creation
    Cart* p2 = new Cart(1000);
    p2->display();

    delete p2;
```

```
    return 0;
}
```

Output



Program Code3 : Array of Pointers to Cart Objects

```
#include <iostream>
using namespace std;

class Cart {
    double total;

public:
    Cart(double t) {
        total = t;
    }

    double getTotal() const {
        return total;
    }

    void display() const {
        cout << "Cart Total: Rs. " << total << endl;
    }
};

int main() {
    Cart c1(500);
```

```
    Cart c2(1200);
    Cart c3(800);

    Cart* carts[3] = {&c1, &c2, &c3};

    int highest = 0;

    cout << "Customer Cart Details" << endl;

    for (int i = 0; i < 3; i++) {
        carts[i]->display();

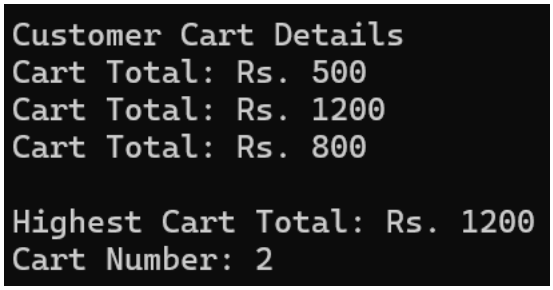
        if (carts[i]->getTotal() >
            carts[highest]->getTotal()) {
            highest = i;
        }
    }

    cout << "\nHighest Cart Total: Rs. "
         << carts[highest]->getTotal() << endl;

    cout << "Cart Number: " << highest + 1 << endl;

    return 0;
}
```

Output



Question 6: Design Challenge – Vehicle Rental Fleet

6.1 Identify the Entities and Their Responsibilities

The vehicle rental company manages different types of vehicles, including bicycles, two-wheelers, cars, and electric cars. Each vehicle has common details, but its rental charges and additional properties differ.

Entity	Attributes	Responsibilities
Vehicle	Registration number, daily rate	Rent, return, calculate charges
Bicycle	Registration number, daily rate	Calculate bicycle rental charge
MotorVehicle	Fuel/charge level, service record	Maintain motor vehicle details
TwoWheeler	Fuel level	Calculate two-wheeler rental charge
Car	Seating capacity	Calculate car rental charge
ElectricCar	Battery capacity, energy used	Calculate electricity-based charge
ElectricBus	Battery capacity, seating capacity	Calculate electric bus rental charge

6.2 Proposed Class Hierarchy

An abstract base class named Vehicle is created to store common vehicle information. Different vehicle classes inherit from it and implement their own rental charge calculations.

Inheritance relationships:

- Single inheritance: Bicycle inherits from Vehicle.
- Hierarchical inheritance: Bicycle and MotorVehicle inherit from Vehicle.
- Multilevel inheritance: Vehicle → MotorVehicle → Car → ElectricCar → ElectricBus.
- Hybrid inheritance: The complete design combines different inheritance relationships.
- Composition: MotorVehicle contains a ServiceRecord object.

6.3 Class Responsibilities and Virtual Functions

The Vehicle class acts as the common base class and contains registration number, daily rental rate, and rental status.

The calculateCharge() function is declared as a pure virtual function. Each derived class provides its own implementation based on its vehicle type.

A virtual destructor is included to ensure proper destruction when derived objects are deleted through base-class pointers.

The ServiceRecord class stores the service due date and is included as a member object inside MotorVehicle.

6.4 Vehicle Rental and Return Operations

Renting operation:

1. Select the required vehicle.
2. Check whether it is available.
3. If available, mark it as rented.
4. Store the rental duration.

Returning operation:

1. Check whether the vehicle is currently rented.
2. Calculate the rental charge using the derived-class function.
3. Add the late-return charge, if applicable.
4. Mark the vehicle as available.

Pseudocode:

START

Create vehicle objects
Store vehicle addresses in an array of Vehicle pointers

Display all vehicle details

FOR each vehicle
  Rent vehicle for selected days
  Calculate rental charge
  Add late-return charge
  Return vehicle
END FOR

Display final rental details

Delete all dynamically allocated objects

STOP

6.5 Expected Design and Testing

The program is tested using different vehicle types.

Vehicle	Rental charge calculation
Bicycle	Daily rate × rental days
Two-wheeler	Daily rate × days + fuel surcharge
Car	Daily rate × days + seating surcharge
Electric car	Daily rate × days + electricity cost
Electric bus	Daily rate × days + electricity cost + seating surcharge

A late-return charge is added to every vehicle's rental cost.

6.6 Alternative Design Approach

Another approach is to create a single Vehicle class with a vehicle-type variable and use switch statements to calculate charges.

However, this approach becomes difficult to maintain when new vehicle types are introduced. Every new type may require changes to the existing class.

Using inheritance and virtual functions is better because new vehicle types can be added by creating new derived classes without modifying existing vehicle classes.

Conclusion

This vehicle rental system demonstrates abstraction, inheritance, polymorphism, composition, and dynamic memory management. The design supports different vehicle types and allows new vehicles to be added easily. It also provides a reusable and extensible structure for managing rental operations.

Program Code1: Complete Vehicle Rental Fleet Management System

```
#include <iostream>

#include <string>

using namespace std;



// Member class for service information

class ServiceRecord {

    string serviceDate;



public:

    ServiceRecord(string date) {

        serviceDate = date;

    }



    void display() const {

        cout << "Service Due: " << serviceDate << endl;

    }

};
```

```
// Abstract base class
class Vehicle {
protected:
    string registration;
    double dailyRate;
    bool rented;
    int rentalDays;

public:
    Vehicle(string reg, double rate)
        : registration(reg), dailyRate(rate),
          rented(false), rentalDays(0) {
    }

    virtual double calculateCharge(int days) const = 0;

    virtual void displayDetails() const {
        cout << "Registration: " << registration << endl;
        cout << "Daily Rate: Rs. " << dailyRate << endl;
    }

    void rent(int days) {
        if (!rented) {
            rented = true;
            rentalDays = days;
```

```
            cout << registration << " rented successfully." << endl;
        } else {
            cout << registration << " is already rented." << endl;
        }
    }

    void returnVehicle(int lateDays) {
        if (rented) {
            double charge = calculateCharge(rentalDays);
            double lateCharge = lateDays * 100;

            cout << "Rental Charge: Rs. " << charge << endl;
            cout << "Late Charge: Rs. " << lateCharge << endl;
            cout << "Total Charge: Rs. "
                 << charge + lateCharge << endl;

            rented = false;
            rentalDays = 0;
        } else {
            cout << registration << " was not rented." << endl;
        }
    }

    virtual ~Vehicle() {
    }
};
```

```
// MotorVehicle class
class MotorVehicle : public Vehicle {
protected:
    double energyLevel;
    ServiceRecord service;

public:
    MotorVehicle(string reg, double rate,
                 double energy, string date)
        : Vehicle(reg, rate),
          energyLevel(energy), service(date) {
    }

    void displayDetails() const override {
        Vehicle::displayDetails();
        cout << "Fuel/Charge Level: "
             << energyLevel << endl;
        service.display();
    }
};

// Bicycle class
class Bicycle : public Vehicle {
public:
    Bicycle(string reg, double rate)
```

```
        : Vehicle(reg, rate) {
    }

    double calculateCharge(int days) const override {
        return dailyRate * days;
    }
};

// TwoWheeler class
class TwoWheeler : public MotorVehicle {
public:
    TwoWheeler(string reg, double rate,
               double fuel, string date)
        : MotorVehicle(reg, rate, fuel, date) {
    }

    double calculateCharge(int days) const override {
        return dailyRate * days + 50;
    }
};

// Car class
class Car : public MotorVehicle {
protected:
    int seats;
```

```
public:
    Car(string reg, double rate, double fuel,
        string date, int capacity)
        : MotorVehicle(reg, rate, fuel, date),
          seats(capacity) {
    }

    double calculateCharge(int days) const override {
        return dailyRate * days + seats * 20;
    }

    void displayDetails() const override {
        MotorVehicle::displayDetails();
        cout << "Seating Capacity: " << seats << endl;
    }
};

// ElectricCar class
class ElectricCar : public Car {
protected:
    double batteryCapacity;
    double energyUsed;
    double costPerKwh;

public:
    ElectricCar(string reg, double rate,
```

```
            double charge, string date,
            int capacity, double battery,
            double used, double cost)
        : Car(reg, rate, charge, date, capacity),
          batteryCapacity(battery),
          energyUsed(used), costPerKwh(cost) {
    }

    double calculateCharge(int days) const override {
        return dailyRate * days
             + energyUsed * costPerKwh;
    }

    void displayDetails() const override {
        Car::displayDetails();
        cout << "Battery Capacity: "
            << batteryCapacity << " kWh" << endl;
    }
};

// New vehicle type added without modifying old classes
class ElectricBus : public ElectricCar {
public:
    ElectricBus(string reg, double rate,
            double charge, string date,
            int capacity, double battery,
```

```
            double used, double cost)
        : ElectricCar(reg, rate, charge, date,
                    capacity, battery, used, cost) {
    }

    double calculateCharge(int days) const override {
        return dailyRate * days
             + energyUsed * costPerKwh
             + seats * 10;
    }
};

int main() {

    // Array of base-class pointers
    Vehicle* fleet[5];

    fleet[0] = new Bicycle("BIKE101", 100);

    fleet[1] = new TwoWheeler(
        "TWO201", 300, 80, "15-12-2026"
    );

    fleet[2] = new Car(
        "CAR301", 1000, 60,
        "20-12-2026", 5
```

```
);

fleet[3] = new ElectricCar(
    "EV401", 1200, 90,
    "25-12-2026", 5, 60, 15, 8
);

fleet[4] = new ElectricBus(
    "BUS501", 3000, 95,
    "30-12-2026", 30, 150, 40, 8
);

cout << "VEHICLE RENTAL FLEET" << endl;

for (int i = 0; i < 5; i++) {
    cout << "\n----------------------" << endl;
    fleet[i]->displayDetails();

    fleet[i]->rent(2);

    cout << "Return Details:" << endl;
    fleet[i]->returnVehicle(1);
}

// Release dynamically allocated memory
for (int i = 0; i < 5; i++) {
```

```
        delete fleet[i];
    }

    return 0;
}
```

Output

```
VEHICLE RENTAL FLEET

----------------------
Registration: BIKE101
Daily Rate: Rs. 100
BIKE101 rented successfully.
Return Details:
Rental Charge: Rs. 200
Late Charge: Rs. 100
Total Charge: Rs. 300

----------------------
Registration: TWO201
Daily Rate: Rs. 300
Fuel/Charge Level: 80
Service Due: 15-12-2026
TWO201 rented successfully.
Return Details:
Rental Charge: Rs. 650
Late Charge: Rs. 100
Total Charge: Rs. 750

----------------------
Registration: CAR301
Daily Rate: Rs. 1000
Fuel/Charge Level: 60
Service Due: 20-12-2026
Seating Capacity: 5
CAR301 rented successfully.
Return Details:
Rental Charge: Rs. 2100
Late Charge: Rs. 100
Total Charge: Rs. 2200

----------------------
Registration: EV401
Daily Rate: Rs. 1200
Fuel/Charge Level: 90
Service Due: 25-12-2026
Seating Capacity: 5
Battery Capacity: 60 kWh
EV401 rented successfully.
Return Details:
Rental Charge: Rs. 2520
Late Charge: Rs. 100
Total Charge: Rs. 2620

----------------------
Registration: BUS501
Daily Rate: Rs. 3000
Fuel/Charge Level: 95
Service Due: 30-12-2026
Seating Capacity: 30
Battery Capacity: 150 kWh
BUS501 rented successfully.
Return Details:
Rental Charge: Rs. 6620
Late Charge: Rs. 100
Total Charge: Rs. 6720
```

Program Code2: Demonstrating Runtime Polymorphism

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    virtual double calculateCharge(int days) const = 0;

    virtual ~Vehicle() {
    }
};

class Bicycle : public Vehicle {
public:
    double calculateCharge(int days) const override {
        return 100 * days;
    }
};

class Car : public Vehicle {
public:
    double calculateCharge(int days) const override {
        return 1000 * days;
    }
};

int main() {
    Bicycle b;
    Car c;

    Vehicle* v[2] = {&b, &c};
```

```
    cout << "Bicycle Charge: Rs. "
         << v[0]->calculateCharge(2) << endl;

    cout << "Car Charge: Rs. "
         << v[1]->calculateCharge(2) << endl;

    return 0;
}
```

Output

